

Definition 1 ($\text{LRA}(G)$). Let $G \triangleq (N, T, P, S)$ be a context-free grammar.

- (i) $G' := (N', T', P', S') \triangleq (N \uplus \{S'\}, T \uplus \{\$, \}, P \cup \{[S' \rightarrow S\$]\}, S')$.
- (ii) $V \triangleq N \uplus T$. $V' \triangleq N' \uplus T'$.
- (iii) $\beta \xRightarrow{\text{rm}}_{P'} \gamma \xLeftrightarrow{\text{def}} (\beta, \gamma) \in \{(\alpha Az, \alpha \omega z) \mid [A \rightarrow \omega] \in P', \alpha \in V'^*, z \in T'^*\}$.
- (iv) $I_{\text{LR}(0)} \triangleq \{[A \rightarrow \alpha \cdot \beta] \mid [A \rightarrow \alpha \beta] \in P'\}$.
- (v) $\text{Cl}(q) =_{\mu} q \cup \{[A \rightarrow \varepsilon \cdot \omega] \mid [A \rightarrow \omega] \in P', [B \rightarrow \beta \cdot A\gamma] \in \text{Cl}(q)\}$ for $q \in \mathcal{P}(I_{\text{LR}(0)})$.
- (vi) $q_0 := \text{Cl}(\{[S' \rightarrow \varepsilon \cdot S\$]\})$.
- (vii) $\text{GOTO}(q, X) := \text{Cl}(\{[A \rightarrow \alpha X \cdot \beta] \mid [A \rightarrow \alpha \cdot X\beta] \in q\})$ for $q \in \mathcal{P}(I_{\text{LR}(0)})$ and $X \in V'$.
- (viii) $Q \triangleq \text{PT} \setminus \{\emptyset\}$ where $\text{PT} =_{\mu} \{q_0\} \cup \{\text{GOTO}(q, X) \mid q \in \text{PT}, X \in V'\}$.
- (ix) $\varepsilon : p \rightarrow q \xLeftrightarrow{\text{def}} p \in Q \wedge p = q$. $X\alpha : p \rightarrow q \xLeftrightarrow{\text{def}} p \in Q \wedge \alpha : \text{GOTO}(p, X) \rightarrow q$.
- (x) **Config** $\triangleq \{(\alpha : p \rightarrow q, z) \mid \alpha \in V'^*, p \in Q, q \in Q, z \in T'^*, \alpha : p \rightarrow q\}$.
- (xi) $\delta(q, X) := \text{GOTO}(q, X)$ for $q \in Q$ and $X \in V'$.
- (xii) **reduce** $(q, t) := \{[A \rightarrow \omega] \mid [A \rightarrow \omega \cdot \varepsilon] \in q\}$ for $q \in Q$ and $t \in T'$.
- (xiii) Let $\vdash$ be a binary relation on the set **Config** with two introduction rules:

$$\frac{q'' = \delta(q', t)}{(\alpha : q \rightarrow q', tz) \vdash (\alpha t : q \rightarrow q'', z)} \text{ Shift}$$

$$\frac{\alpha : q \rightarrow p \quad \omega : p \rightarrow q' \quad [A \rightarrow \omega] \in \text{reduce}(q', t) \quad q'' = \delta(p, A)}{(\alpha \omega : q \rightarrow q', tz) \vdash (\alpha A : q \rightarrow q'', tz)} \text{ Reduce}(A \rightarrow \omega)$$

- (xiv) $q_f := \delta(\delta(q_0, S), \$)$.
- (xv) The language accepted by $\text{LRA}(G)$ is defined as follows:

$$L(\text{LRA}(G)) := \{z \in T^* \mid (\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon)\}.$$

Fact 2. *It is known that $L(G) = L(\text{LRA}(G))$, where $L(G) \triangleq \{z \in T^* \mid S \Rightarrow_P^* z\}$.*

Proof. We first record the standard viable-prefix property of the $\text{LR}(0)$ automaton. If η is a viable prefix, then there is a state $q \in Q$ such that $\eta : q_0 \rightarrow q$. Moreover, if an occurrence of ω at the end of η is a handle for a production $[A \rightarrow \omega]$, then the state reached by η contains the item $[A \rightarrow \omega \cdot \varepsilon]$. This is an immediate induction on the construction of Cl , GOTO , and paths in the characteristic $\text{LR}(0)$ automaton.

For soundness, suppose that

$$(\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon).$$

For a configuration $(\alpha : p \rightarrow q, u)$, call αu its yield. A shift step preserves the yield. A reduce step

$$(\alpha \omega : q \rightarrow q', tz) \vdash (\alpha A : q \rightarrow q'', tz)$$

with $[A \rightarrow \omega] \in P'$ changes the yield from $\alpha \omega tz$ to αAtz , and the latter derives the former by one rightmost step:

$$\alpha Atz \xRightarrow{\text{rm}}_{P'} \alpha \omega tz,$$

because the suffix tz is terminal. Therefore, reading the accepting computation backwards and ignoring the shift steps, we obtain

$$S\$ \xRightarrow{\text{rm}}_{P'}^* z\$.$$

Since $\$ \notin T$ and the only role of the augmented production $[S' \rightarrow S\$]$ is to create the initial and final $\text{LR}(0)$ states, this is equivalent to $S \Rightarrow_P^* z$. Hence $L(\text{LRA}(G)) \subseteq L(G)$.

For completeness, assume $S \Rightarrow_P^* z$. Then $S\$ \xRightarrow{\text{rm}}_{P'}^* z\$$. Choose such a rightmost derivation and run it backwards. Whenever the reverse derivation exposes a terminal as the next unread input symbol, use the Shift rule. Whenever the reverse derivation replaces a handle ω by its left-hand side A , say

$$\alpha Atz \xRightarrow{\text{rm}}_{P'} \alpha \omega tz,$$

the viable-prefix property gives states $p, q \in Q$ with $\alpha : q_0 \rightarrow p$, $\omega : p \rightarrow q$, and $[A \rightarrow \omega \cdot \varepsilon] \in q$. Thus $[A \rightarrow \omega] \in \mathbf{reduce}(q, t)$, and the Reduce rule performs exactly the inverse step from $(\alpha\omega : q_0 \rightarrow q, tz)$ to $(\alpha A : q_0 \rightarrow \delta(p, A), tz)$. Induction over the reversed rightmost derivation yields

$$(\varepsilon : q_0 \rightarrow q_0, z\$) \vdash^* (S\$: q_0 \rightarrow q_f, \varepsilon).$$

Thus $L(G) \subseteq L(\text{LRA}(G))$, and the two languages are equal. $\square$

Theorem 3. Define $\mathbf{LA} : \{(q, [A \rightarrow \omega]) \mid q \in Q, [A \rightarrow \omega \cdot \varepsilon] \in q\} \rightarrow \mathcal{P}(T')$ by

$$\mathbf{LA}(q, [A \rightarrow \omega]) := \left\{ t \in T' \mid S' \xRightarrow[\mathbf{rm}_{P'}]{*} \alpha A t z, \alpha\omega : q_0 \rightarrow q, \alpha \in V'^*, z \in T'^* \right\}. \quad (1)$$

Then, overriding $\mathbf{reduce} : Q \times T' \rightarrow \mathcal{P}(P')$ of the LR(0) parser $\text{LRA}(G)$ with

$$(q, t) \mapsto \{[A \rightarrow \omega] \mid [A \rightarrow \omega \cdot \varepsilon] \in q, t \in \mathbf{LA}(q, [A \rightarrow \omega])\}$$

yields an LALR(1) parser if there are no conflicts, which accepts the same language.

Proof. Let $\vdash_{\mathbf{LA}}$ denote the transition relation obtained from $\vdash$ by replacing $\mathbf{reduce}$ with the lookahead-restricted function displayed in the statement. Every $\vdash_{\mathbf{LA}}$ -reduction is also an LR(0) reduction, so every $\vdash_{\mathbf{LA}}$ -computation is an $\text{LRA}(G)$ -computation. By Fact 2, this gives

$$L(\vdash_{\mathbf{LA}}) \subseteq L(\text{LRA}(G)) = L(G).$$

Conversely, let $z \in L(G)$. In the completeness part of the proof of Fact 2 we constructed an accepting computation by reversing a rightmost derivation

$$S\$ \xRightarrow[\mathbf{rm}_{P'}]{*} z\$.$$

Consider any reduction used in that computation. It is the inverse of a rightmost step

$$\alpha A t z \xRightarrow[\mathbf{rm}_{P'}]{*} \alpha\omega t z$$

for some $t \in T'$ and $z \in T'^*$, and at the moment of the reduction the stack prefix is $\alpha\omega : q_0 \rightarrow q$, where $[A \rightarrow \omega \cdot \varepsilon] \in q$. Hence, by the definition of $\mathbf{LA}$,

$$t \in \mathbf{LA}(q, [A \rightarrow \omega]).$$

Therefore every reduction in that accepting computation is still permitted by $\vdash_{\mathbf{LA}}$. The same computation accepts z , so $L(G) \subseteq L(\vdash_{\mathbf{LA}})$. Hence the lookahead-restricted parser accepts exactly $L(G)$.

The action table of this parser consists of the LR(0) shift actions together with precisely those reduce actions whose terminal lookahead belongs to the corresponding $\mathbf{LA}$ -set. Thus it is the usual LALR(1) table obtained from the LR(0) cores by attaching LALR(1) lookahead sets to reduce items. If no state/lookahead pair contains two enabled actions, namely no shift/reduce or reduce/reduce conflict, the table is deterministic with one symbol of lookahead; hence it yields an LALR(1) parser. $\square$

Theorem 4. Letting $R!x \triangleq \{y \mid (x, y) \in R\}$, define relations **Read** and **Follow** from the set

$$\{(p, A) \mid p \in Q, A \in N', \delta(p, A) \neq \emptyset\}$$

to the set T' inductively as follows:

$$\begin{array}{l} \frac{\delta(\delta(p, A), t) \neq \emptyset}{t \in \mathbf{Read}!(p, A)} \text{DR} \qquad \frac{\delta(p, A) = r \quad C \Rightarrow_{P'}^* \varepsilon}{\mathbf{Read}!(r, C) \subseteq \mathbf{Read}!(p, A)} \text{reads} \\[10pt] \frac{t \in \mathbf{Read}!(p, A)}{t \in \mathbf{Follow}!(p, A)} \text{Read} \qquad \frac{[B \rightarrow \beta \cdot A\gamma] \in p \quad \beta : p' \rightarrow p \quad \gamma \Rightarrow_{P'}^* \varepsilon}{\mathbf{Follow}!(p', B) \subseteq \mathbf{Follow}!(p, A)} \text{includes} \end{array}$$

Then, whenever $q \in Q$ and $[A \rightarrow \omega \cdot \varepsilon] \in q$, we can compute $\mathbf{LA}(q, [A \rightarrow \omega])$ by

$$\mathbf{LA}(q, [A \rightarrow \omega]) = \{t \in T' \mid p \in Q, \omega : p \rightarrow q, \delta(p, A) \neq \emptyset, t \in \mathbf{Follow}!(p, A)\}. \quad (2)$$

Proof. Let

$$D = \{(p, A) \mid p \in Q, A \in N', \delta(p, A) \neq \emptyset\}.$$

For the proof, define two semantic sets on D . First, let

$$\text{Read}(p, A) = \{t \in T' \mid \delta(p, A) = r, \gamma \Rightarrow_{P'}^* \varepsilon, \gamma t : r \rightarrow s \text{ for some } \gamma \in N'^*, s \in Q\}.$$

Thus $t \in \text{Read}(p, A)$ exactly when, after the transition on A , one can reach a terminal transition on t through zero or more nullable nonterminal transitions. Second, let

$$\text{Follow}(p, A) = \left\{ t \in T' \mid S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z, \alpha : q_0 \rightarrow p, \alpha \in V'^*, z \in T'^* \right\}.$$

This is the semantic follow set of the nonterminal transition (p, A) .

We first show that the displayed rules compute **Read**. If $\gamma = \varepsilon$ in the definition of $\text{Read}(p, A)$, then $\delta(\delta(p, A), t) \neq \emptyset$, which is exactly the rule DR. If $\gamma = C\gamma'$, then $C \Rightarrow_{P'}^* \varepsilon$, $\delta(p, A) = r$, and $\gamma't : \delta(r, C) \rightarrow s$. Hence $t \in \text{Read}(r, C)$, and the rule reads propagates this terminal to $\text{Read}(p, A)$. The converse direction is immediate from the same decomposition: DR corresponds to $\gamma = \varepsilon$, and each use of reads prefixes one nullable nonterminal to γ . Since D and T' are finite, the inductive definition is the least fixed point of these monotone clauses, so

$$\mathbf{Read}!(p, A) = \text{Read}(p, A) \quad \text{for all } (p, A) \in D.$$

When $(r, C) \notin D$, the set $\mathbf{Read}!(r, C)$ is empty, so the printed reads rule is vacuous in that case.

Next we characterize **Follow**. The first terminal after an occurrence of A in a right-sentential form can arise in exactly two ways.

First, it can be read inside the same local LR(0) context after the transition on A , possibly after passing through nullable nonterminals. These terminals are precisely $\text{Read}(p, A)$, and hence they are included in $\text{Follow}(p, A)$ by the rule Read.

Second, the occurrence of A can appear in a production $[B \rightarrow \beta A \gamma]$ whose suffix γ is nullable. If $[B \rightarrow \beta \cdot A \gamma] \in p$, $\beta : p' \rightarrow p$, and $\gamma \Rightarrow_{P'}^* \varepsilon$, then every terminal that can follow the transition (p', B) can also follow (p, A) : from a sentential form $\alpha B t z$ with $\alpha : q_0 \rightarrow p'$, one rightmost expansion of B gives $\alpha \beta A \gamma t z$, and then $\gamma \Rightarrow^* \varepsilon$ gives a right-sentential form whose prefix $\alpha \beta$ reaches p and in which t follows A . This is exactly the rule includes.

Conversely, given $S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z$ with $\alpha : q_0 \rightarrow p$, inspect the production occurrence in the rightmost derivation that produced the displayed occurrence of A . If the first terminal t after A lies inside the remaining suffix of that production after passing only through nullable nonterminals, then $t \in \text{Read}(p, A)$. Otherwise the entire suffix to the right of A in that production is nullable, and the terminal t is inherited from the symbol on the left-hand side of that production; this gives an includes edge from some (p', B) to (p, A) . Repeating this argument moves outward through the finite rightmost derivation until a **Read**-case is reached. Therefore **Follow** is the least solution of the Read and includes clauses, and so

$$\mathbf{Follow}!(p, A) = \text{Follow}(p, A) \quad \text{for all } (p, A) \in D.$$

It remains to translate this semantic follow set into the lookahead set of a completed item. Let $q \in Q$ and $[A \rightarrow \omega \cdot \varepsilon] \in q$. Then

$$\begin{aligned} t \in \mathbf{LA}(q, [A \rightarrow \omega]) & \\ \iff \exists \alpha, z. S' \xRightarrow[\text{rm}_{P'}]^* \alpha A t z \wedge \alpha \omega : q_0 \rightarrow q & \\ \iff \exists p \in Q, \omega : p \rightarrow q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}(p, A) & \\ \iff \exists p \in Q, \omega : p \rightarrow q \wedge \delta(p, A) \neq \emptyset \wedge t \in \mathbf{Follow}!(p, A). & \end{aligned}$$

The middle equivalence just cuts the path $\alpha \omega : q_0 \rightarrow q$ at the point reached after α ; conversely, concatenating $\alpha : q_0 \rightarrow p$ and $\omega : p \rightarrow q$ recovers $\alpha \omega : q_0 \rightarrow q$. This is precisely the formula claimed in the theorem. $\square$

Appendix A. Digraph algorithm and implementation specification

This appendix gives an executable specification for computing the sets used in Theorem 4. The construction is the usual digraph view of LALR(1) lookahead computation: direct seed sets are placed on nodes, dependency edges describe set inclusions, and the least fixed point is obtained by collapsing strongly connected components and propagating set unions along the resulting DAG [1, 2, 3].

A.1. General digraph fixed-point problem

Let X be a finite set of nodes, let $E \subseteq X \times X$ be a directed edge relation, and let $\text{Seed} : X \rightarrow \mathcal{P}(T')$. We use the edge orientation

$$x \rightsquigarrow y \quad \text{to mean} \quad F(y) \subseteq F(x).$$

Thus the desired solution is the least function $F : X \rightarrow \mathcal{P}(T')$ satisfying

$$F(x) = \text{Seed}(x) \cup \bigcup_{x \rightsquigarrow y} F(y). \quad (\text{DG})$$

Algorithm DIGRAPH(X, E, Seed).

1. Compute the strongly connected components of (X, E) , for example by Tarjan's depth-first-search algorithm.
2. Build the condensation DAG $\mathcal{C}$. Its nodes are SCCs, and $C \rightsquigarrow D$ is an edge when some $x \in C$ and $y \in D$ satisfy $x \rightsquigarrow y$ and $C \neq D$.
3. Initialize

$$\text{Val}(C) := \bigcup_{x \in C} \text{Seed}(x)$$

for every SCC C .

4. Process the SCCs in reverse topological order with respect to $\rightsquigarrow$, so that every successor D of C has already been processed, and put

$$\text{Val}(C) \leftarrow \text{Val}(C) \cup \bigcup_{C \rightsquigarrow D} \text{Val}(D).$$

5. Return $F(x) := \text{Val}(C_x)$, where C_x is the SCC containing x .

Correctness condition. For every SCC C , after step 4,

$$\text{Val}(C) = \bigcup \{ \text{Seed}(y) \mid \text{some } x \in C \text{ reaches } y \text{ in } (X, E) \}.$$

Therefore the returned F is exactly the least solution of (DG). This is the only property of the graph algorithm needed by Theorem 4.

Complexity. Let $b = \lceil |T'|/w \rceil$, where w is the machine word size used for bitsets. SCC construction is $O(|X| + |E|)$, and the set propagation is $O((|X| + |E|)b)$. A simple work-list implementation that repeatedly applies $F(x) := F(x) \cup F(y)$ for $x \rightsquigarrow y$ is also correct, but the SCC version avoids repeated propagation inside cycles.

A.2. Instance for Read

Let

$$D = \{(p, A) \mid p \in Q, A \in N', \delta(p, A) \neq \emptyset\}.$$

First compute nullable nonterminals by the least fixed point

$$\text{Null}(A) \iff \exists [A \rightarrow X_1 \cdots X_k] \in P' \text{ such that every } X_i \text{ is nullable,}$$

where terminals are never nullable and $k = 0$ is allowed. Extend this to strings by $\text{Null}(X_1 \cdots X_k)$ iff every X_i is nullable.

For each node $x = (p, A) \in D$, define the direct-read seed

$$\text{DR}(p, A) = \{t \in T' \mid \delta(\delta(p, A), t) \neq \emptyset\}.$$

Define the reads dependency graph $E_R \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_R (r, C) \iff \delta(p, A) = r \wedge C \in N' \wedge \text{Null}(C) \wedge (r, C) \in D.$$

The last condition merely makes explicit the domain of Theorem 4. If $(r, C) \notin D$, then $\mathbf{Read}!(r, C) = \emptyset$, so no edge is needed.

Then compute

$$\mathbf{Read}!(p, A) := \text{DIGRAPH}(D, E_R, \text{DR})(p, A).$$

This is the least relation satisfying the DR and reads rules of Theorem 4.

A.3. Instance for Follow

The seed for follow propagation is the already computed read set:

$$\text{Seed}_F(p, A) := \mathbf{Read}!(p, A).$$

Define the includes dependency graph $E_I \subseteq D \times D$ by

$$(p, A) \rightsquigarrow_I (p', B) \iff [B \rightarrow \beta \cdot A\gamma] \in p \wedge \beta : p' \rightarrow p \wedge \text{Null}(\gamma) \wedge (p', B) \in D.$$

The edge orientation again means that the right-hand node contributes to the left-hand node:

$$\mathbf{Follow}!(p', B) \subseteq \mathbf{Follow}!(p, A).$$

Then compute

$$\mathbf{Follow}!(p, A) := \text{DIGRAPH}(D, E_I, \text{Seed}_F)(p, A).$$

This is the least relation satisfying the Read and includes rules of Theorem 4.

A.4. Lookback relation and lookahead construction

For every completed item $[A \rightarrow \omega \cdot \varepsilon] \in q$, define its lookback set by

$$\text{LB}(q, A \rightarrow \omega) := \{(p, A) \in D \mid \omega : p \rightarrow q\}.$$

The lookahead set is then computed by

$$\mathbf{LA}(q, [A \rightarrow \omega]) := \bigcup_{(p, A) \in \text{LB}(q, A \rightarrow \omega)} \mathbf{Follow}!(p, A).$$

This is the formula of Theorem 4 written as an implementation step.

Implementation note. The query $\omega : p \rightarrow q$ can be implemented by following δ -edges from p along ω . The query $\beta : p' \rightarrow p$ in E_I can be implemented by caching reverse-path queries keyed by (β, p) , or by storing predecessor edges during construction of the LR(0) automaton.

A.5. Parser-table and conflict specification

After **LA** has been computed, construct actions as follows.

1. If $\delta(q, t) \neq \emptyset$ for $t \in T'$, add the shift action $q \xrightarrow{t} \delta(q, t)$.
2. For each $[A \rightarrow \omega \cdot \varepsilon] \in q$ and each $t \in \mathbf{LA}(q, [A \rightarrow \omega])$, add the reduce action $A \rightarrow \omega$ under lookahead t .
3. Report a shift/reduce conflict when both a shift action and a reduce action are present for the same pair (q, t) .
4. Report a reduce/reduce conflict when two distinct reduce productions are present for the same pair (q, t) .

The accepting condition remains the one in Definition 1: the parser accepts when it reaches $(S\$: q_0 \rightarrow q_f, \varepsilon)$. Equivalently, in a conventional ACTION/GOTO table, q_f may be treated as the accepting state after the input marker has been consumed.

A.6. Summary specification

The complete computation required by Theorem 4 is:

1. Build G' , Q , and δ as in Definition 1.
2. Compute nullable nonterminals and nullable strings.
3. Build D , $\mathbf{DR}$, and E_R ; compute **Read** by $\mathbf{DIGRAPH}(D, E_R, \mathbf{DR})$.
4. Build E_I ; compute **Follow** by $\mathbf{DIGRAPH}(D, E_I, \mathbf{Read})$.
5. Build **LB** and compute each $\mathbf{LA}(q, [A \rightarrow \omega])$ by unioning the appropriate **Follow**-sets.
6. Construct the guarded reduce actions and check conflicts.

The postcondition is that the computed **Read**, **Follow**, and **LA** are the least sets satisfying Theorem 4, and hence the lookahead-restricted parser of Theorem 3 uses exactly the LALR(1) lookahead sets determined by the LR(0) automaton.

References

- [1] F. DeRemer and T. J. Pennello. Efficient computation of LALR(1) look-ahead sets. *ACM Transactions on Programming Languages and Systems*, 4(4):615–649, 1982.
- [2] R. E. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [3] J. H. Gallier. A survey of LR-parsing methods: The graph method for computing fixed points and LALR(1) lookahead sets. CIS 511 lecture notes, University of Pennsylvania, 2008.